Adaptive Retransmission Scheduling in Underwater Acoustic Sensor Networks based on ARQ_HMAC

Team Members

1. 241CS209, Ankit Kumar, ankitkumar.241cs209@nitk.edu.in, 7295030513
2. 241CS239, Navadeep Sai K, navadeepsaikancharla.241cs239@nitk.edu.in, 8310029882
3. 241CS255, Shivam Kumar, shivam.241cs255@nitk.edu.in, 9234074220

Abstract

Ocean monitoring, environmental research, deep-sea exploration, and surveillance all rely on Underwater Acoustic Sensor Networks (UASNs) to function. Because acoustic waves move slowly and the available bandwidth is quite narrow, underwater links end up dealing with very large propagation delays [2, 3, 4, 5]. On top of that, whenever two or more nodes try to use the channel simultaneously, their packets crash into each other and have to be sent again, a process that burns through time, energy, and whatever little channel capacity exists [3, 5, 9]. Our work takes ARQ_HMAC [1] as a starting point. This is a hybrid MAC protocol that blends contention-based access with Automatic Repeat reQuest (ARQ) and pushes failed packets into collision-free TDMA slots for retransmission. The catch is that ARQ_HMAC measures each node's propagation delay a single time during setup and then pads everything with a large fixed guard interval to stay safe against timing drift. But underwater conditions shift constantly and nodes move around after deployment, so that initial measurement quickly goes stale and the oversized guard ends up wasting valuable channel time [3, 7]. What we propose, called Adaptive ARQ_HMAC, fixes this by continuously refreshing the delay estimate using an exponentially weighted moving average paired with a change-detection threshold. It extracts timing information from time-tagged control frames, determines the size of the guard interval based on delay variance, schedules the TDMA slots based on actual packet arrivals, and fine-tunes the back-off algorithm according to the current level of contention. We coded both the original and the adaptive version inside the UnetStack simulator and ran 240 simulations across three scenarios: a static network, a 400 m network with moving AUVs, and a 3 km network with fast-moving AUVs. The adaptive technique leads to a reduction in the end-to-end delay by about 65–72%, reduces the number of retransmissions by 44–47%, collision rate by 23–30%, while improving throughput by 6–23% and packet delivery ratio by 3–12%. All these benefits have statistical significance ($p < 0.001$).

1 Introduction

1.1 Background

Over the past couple of decades, UASNs have grown into an essential piece of technology for a wide range of tasks: everything from gathering oceanographic data and keeping tabs on marine ecosystems to supporting offshore drilling operations, tsunami warning infrastructure, and naval defense [2, 3]. The normal setup involves dispersing nodes with sensors on the seabed or attaching them to autonomous underwater vehicles (AUVs). The data collected is transmitted to the master node, which in turn transmits the data to a buoy on the surface or a research vessel. The big difference from the terrestrial wireless sensor networks most people are familiar with is the physical layer: radio-frequency (RF) signals get absorbed almost immediately in water, and optical signals scatter too much to be practical, so UASNs have to rely on acoustic waves instead [2, 4].

And the acoustic channel is a very different beast from an RF channel. Sound moves through water at roughly 1500 m/s, about five orders of magnitude slower than electromagnetic waves travel through air. That means propagation delays can stretch from a few tens of milliseconds all the way to several seconds, depending on the distance between nodes [2, 5]. To put things in numerical perspective, by 400 m the delay is approximately 0.27 s and by 3 km it becomes 2 s, which actually exceeds the time needed for a packet transfer onto the link itself. Another challenge is bandwidth. It can be very limited and in most cases equals several kilohertz only, which value decreases even more with increasing range [3, 4]. With data rates hovering in the kilobits-per-second range, every single packet hogs the channel for a noticeable chunk of time. All of this makes designing protocols for UASNs far more challenging than for their terrestrial counterparts [2, 5]. It is the MAC layer that lies at the very core of this issue since it controls access to the common acoustic medium by multiple nodes and has implications for throughput, delays, energy usage, and reliability [4, 5].

1.2 Challenges in Underwater Acoustic Sensor Networks

The main reason building MAC and retransmission protocols for UASNs is so hard comes down to the hostile, ever-shifting nature of the underwater acoustic environment. The single biggest headache is the long and unpredictable propagation delay, which makes it nearly impossible to just port over the MAC or ARQ schemes that work fine on land [2, 8]. Because signals take so long to get from one place to another, there is a huge window of uncertainty where a sender simply has no idea what is going on at the receiver's end [3, 5]. Carrier sensing, which works reasonably well in terrestrial Wi-Fi, becomes far less trustworthy underwater: a node might listen to the channel, hear nothing, and start transmitting, only for its packet to slam into another packet that was already in transit and just hadn't arrived yet. The result is a much higher collision rate than you would see on land [3].

And once collisions start happening, the network has to retransmit, which drains battery power,

inflates delays, and piles more traffic onto an already congested channel [5, 9]. The narrow bandwidth amplifies the pain because each retransmission eats up a disproportionately large slice of the available capacity [2, 4]. Worse still, if the colliding nodes happen to wait the same amount of time before retrying, they will just collide again, potentially locking the network into a repeated failure loop. So the timing of retransmissions matters just as much as the decision to retransmit in the first place [1].

On top of all that, the underwater channel is in a constant state of flux. Temperature gradients, salinity changes, ocean currents, reflections off the surface, and the movement of nodes or even passing marine life all conspire to alter the sound speed profile, multipath structure, and signal attenuation from one moment to the next [3, 7]. These shifts directly affect how long it takes a signal to propagate and how reliable the link is [3, 7]. Mobile platforms like AUVs and underwater gliders can change their distance to the gateway by hundreds of metres in a matter of minutes, meaning a delay measurement taken at deployment time might be completely wrong just a short while later [6]. And since underwater nodes run on batteries that are extremely difficult to swap out or recharge once deployed, squeezing every bit of efficiency out of the channel is not optional; it is a hard requirement [5, 9].

1.3 Existing Approaches and Their Limitations

Researchers have come up with quite a few MAC protocols for UASNs over the years, and they generally fall into three buckets: contention-free, contention-based, and hybrid [4, 5]. Contention-free schemes like TDMA and FDMA hand each node its own dedicated slice of the channel, which sounds clean in theory but in practice leads to poor utilization and demands tight synchronization, something that is notoriously hard to pull off given the enormous delays underwater [2, 4]. Contention-based protocols such as ALOHA and various CSMA flavors are more flexible and dodge the synchronization problem, but they tend to fall apart under heavy traffic because collisions skyrocket [4, 5]. Hybrid protocols try to get the best of both worlds by switching between the two modes depending on what the network looks like at any given moment [1, 4].

One of the more recent entries in this space is ARQ_HMAC [1], which pairs a contention-based CSMA phase for initial transmissions with a TDMA-based retransmission phase for recovery. After a collision, each involved node gets its own TDMA slot to retransmit, which neatly sidesteps the cascading collision problem that plagues purely contention-based designs and keeps throughput from collapsing as the load climbs. That said, ARQ_HMAC, like most protocols in this category, leans on a one-time propagation delay estimate gathered during an initialization handshake. To absorb the inevitable timing errors that creep in later, it slaps on a large fixed guard time ($2t_d + t_{ACK}$) in both its acknowledgement timeout and its TDMA slot structure. The downside is that retransmission rounds drag on longer than they need to, pushing up end-to-end delay.

1.4 Problem Statement

The problem with the ARQ_HMAC technique is that it calculates the propagation delay for each node to the master only once during the initial setup and then uses that static value to determine all subsequent acknowledgments' time-outs and TDMA transmission schedules. However, the underwater environment is dynamic. Channel conditions drift and nodes physically move, so the real delay can wander quite far from whatever was measured at the start. The protocol's way of coping is to build in conservative, fixed timing margins, which unfortunately waste channel time and inflate end-to-end delay. What this work sets out to do is make ARQ_HMAC's retransmission scheduling responsive to the actual, evolving channel, so it stays correct when delays shift and gets faster when the delay is well understood.

1.5 Objectives

This project has three primary goals:

1. Build an adaptive retransmission scheduling mechanism that continuously refreshes the propagation delay estimate using real-time feedback, replacing the single-shot estimation that ARQ_HMAC relies on.

2. Implement both the original ARQ_HMAC protocol and the proposed adaptive variant inside the UnetStack simulation framework.

3. Evaluate and compare the two approaches in terms of Packet Delivery Ratio (PDR), throughput, retransmission count, collision rate, and end-to-end delay across a range of traffic loads and channel conditions.

Apart from the adaptive delay estimation presented in the previous progress report, the completed protocol additionally incorporates the better delay estimate for adjusting the guard time and arrangement of TDMA slots, as well as implements adaptive backoff scheme. All these modifications make use of the enhanced delay estimation for producing tangible gains in performance.

1.6 Organization of the Report

The rest of this report is organized as follows. The Related Work section is covered in Section 2, which will begin with an historical overview in Section 2.1, followed by a more focused discussion of particular algorithms and their strengths and weaknesses in Section 2.2 and finally ending with a direct comparison in Section 2.3. The Proposed Adaptive ARQ_HMAC is discussed in detail in Section 3, which begins with an explanation of the architecture (Section 3.1), then algorithmic discussion (Section 3.2) and finally rationale (Section 3.3). The implementation within the UnetStack framework is discussed in Section 4, and the Results are covered in Section 5. Conclusions are drawn in Section 6.

2 Literature Review

In this section, we briefly review the various research lines in designing MAC and ARQ protocols for underwater acoustic sensor networks. We first present a brief chronology of the major works followed by an in-depth look at some exemplary algorithms.

2.1 Existing Approaches in Chronological Order

Early works (2005–2009). One of the first comprehensive looks at the research landscape for UASNs came from Akyildiz et al. [2], who laid out the fundamental differences between terrestrial and underwater acoustic communication, especially the punishing propagation delays and the scarcity of bandwidth. Around the same time, Stojanovic [10] dug into ARQ for the underwater setting and showed that the classic stop-and-wait approach squanders most of the channel time just sitting around waiting for acknowledgements when delays are long. Her takeaway was that packet sizing and retransmission strategy need to be designed with the delay front and center. In 2006, Molins and Stojanovic [8] introduced Slotted FAMA, among the earliest handshaking MAC protocols tailored for underwater use. In the following year, there were many activities: Park and Rodoplu [9] introduced a MAC protocol called UWAN-MAC based on energy-saving locally-scheduled protocol; Chirdchoo et al. [11] investigated ALOHA-based schemes taking advantage of overheard messages to determine the distance between neighbours; and Peleato and Stojanovic [12] introduced DACAP, a handshaking procedure in which waiting times are adjusted according to distance between nodes. Syed et al. [13] followed up in 2008 with T-Lohi, a contention protocol that fires off short tones to sense and count competing nodes before committing to a data transmission. Rounding out this early period, Molinares and Labrador [7] put forward a logical link layer protocol in 2009 that adjusts its transmission knobs based on observed channel quality.

Intermediate developments (2010–2018). Casari and Zorzi [3] contributed an important theoretical treatment of the spatial-temporal uncertainty inherent in underwater channels and examined which design intuitions from terrestrial networking still hold up. In 2014, Chitre et al. [15] released UnetStack, an agent-based protocol stack and simulator for underwater networks that we ended up using for this project. Also in 2014, Noh et al. [14] proposed DOTS, a delay-aware opportunistic MAC that builds a map of propagation delays to neighbours from overheard traffic and uses that map to schedule concurrent transmissions, even when nodes are mobile. Cho et al. [6] tackled the problem of asymmetric propagation delays in mobile scenarios and designed a TDMA scheme that accounts for the fact that the delay from A to B may differ from B to A. That same year, Jiang [4] published a broad survey of underwater MAC protocols organized around a reference model, noting a clear trend in the field toward hybrid designs.

Recent advances (2019–2025). Kulla et al. [5] delivered an updated survey that zooms in on the trade-offs between contention-based and contention-free approaches. Most recently, Dao and Tran [1] introduced ARQ_HMAC, which serves as the baseline for our work. It merges CSMA-style

contention with TDMA-based collision-free retransmissions and demonstrates higher, more stable throughput than Pure ALOHA, ALOHA with CSMA, Slotted ALOHA, and Slotted ALOHA with CSMA, though it still depends on a static, one-shot delay estimate.

Adaptive timers outside the underwater domain. The problem of setting timeouts when the round-trip time is uncertain and drifting has been studied extensively in terrestrial networking. Jacobson's classic 1988 paper [16] argued that a TCP retransmission timer should track both the average and the variability of the measured round-trip time, an idea that eventually made its way into RFC 6298 [17]. Similarly, the binary exponential backoff mechanism in IEEE 802.11 [18] dynamically adjusts the contention window based on how much contention the node is actually seeing. Both of these ideas find their way into the protocol we propose.

2.2 Representative Algorithms, with Pros and Cons

Here we take a closer look at six representative protocols. For each one, we outline how it works and then weigh its strengths against its weaknesses. The goal is to pinpoint the specific gaps that our proposed mechanism tries to fill.

2.2.1 Slotted FAMA

Slotted Floor Acquisition Multiple Access [8] uses an RTS/CTS protocol where time is divided into slots whose duration corresponds to the maximum propagation delay plus one control packet. Transmissions are only allowed at the beginning of the slot. The node first sends an RTS and awaits a response in the form of a CTS after which it sends the payload data. One benefit of this scheme is that data packets are collision-free as the collision takes place between the control packets. This is illustrated in Algorithm 1 and Table 1 below.

Algorithm 1 Slotted FAMA (sender side)

```
1: wait for the start of the next slot
2: send RTS to the receiver
3: if CTS received within the following slot then
4:     send DATA at the start of the next slot
5:     wait for ACK
6: else
7:     back off for a random number of slots and go to line 1
8: end if
```

Table 1: Advantages and limitations of Slotted FAMA.

Advantages	**Limitations**
Gets rid of data packet collisions entirely and provides reasonably fair channel access among competing nodes. Synchronizing to slot boundaries means the handshake does not depend on knowing the exact propagation delay.	Each slot has to be long enough for the maximum propagation delay, which kills channel utilization. Every data packet triggers a full handshake, an expensive proposition when delays are large, and any retransmission requires going through the whole handshake again.

2.2.2 UWAN-MAC

UWAN-MAC [9] focuses on energy first, in which each node determines its own transmission schedule and shares it with neighbouring nodes, who will remain in sleep mode unless there is a transmission known. It does not use global synchronization but rather focuses on passive channel listening for coordination purposes (refer to Table 2).

Table 2: Advantages and limitations of UWAN-MAC.

Advantages	**Limitations**
Extremely energy-efficient because it cuts idle listening to a minimum and sidesteps the overhead of global clock synchronization.	Has no real mechanism for resolving collisions on the fly: if multiple nodes end up with overlapping schedules, they can keep colliding over and over. There is also no adaptive handling of delay changes for retransmission scheduling.

2.2.3 T-Lohi

In T-Lohi [13], a contention algorithm, the transmitting node sends out a tone when it wishes to send data. The number of tones heard by the nodes will give them an estimate of the number of competing nodes, and they will then defer transmission. These trade-offs have been summarized in Table 3.

Table 3: Advantages and limitations of T-Lohi.

Advantages	**Limitations**
The short tones make contention cheap and give nodes a direct read on the current load, which helps keep data packet collisions down.	Contention rounds need to last at least as long as the maximum propagation delay, which throttles throughput in larger networks. When a packet is lost, it just goes back through the same contention process; there is no dedicated collision-free retransmission phase.

2.2.4 DOTS

DOTS [14] is a delay-sensitive opportunistic MAC. The nodes keep learning about the delays to their neighbors through the use of timestamps received from overheard packets and then use the delay map to schedule transmission times that do not collide with any of the receivers regardless of other ongoing transmissions. Refer to Table 4 below for more details.

Table 4: Advantages and limitations of DOTS.

Advantages	**Limitations**
Uses the lengthy propagation delay to its advantage by allowing concurrent transmissions rather than working against it. The delay map is kept up-to-date using overheard traffic, which can deal with mobility fairly well.	Demands synchronized clocks and a significant amount of per-node state. Scheduling decisions get complicated fast as the neighbour count grows. There is also no structured retransmission phase for when collisions do happen.

2.2.5 Asymmetric Propagation Delay-Aware TDMA

Cho et al. [6] introduced an asymmetric TDMA scheme that takes into consideration the asymmetric nature of the delay encountered while traveling from the sender to the receiver in mobile underwater environments (see Table 5).

Table 5: Advantages and limitations of Asymmetric Delay-Aware TDMA.

Advantages	**Limitations**
Handles mobility and delay asymmetry problem, hence providing more precise scheduling under dynamic environment than the conventional TDMA algorithm.	Delay adjustments happen at fixed intervals, so a sudden change between two updates can still throw off slot alignment. And like any pure TDMA scheme, it leaves slots sitting empty when the traffic load is light.

2.2.6 ARQ_HMAC

ARQ_HMAC [1] is a hybrid MAC that uses asynchronous CSMA for first-time transmissions and switches to synchronous TDMA for retransmissions. During an initialization phase, every node measures its propagation delay t_{D_i} to the master. When the master spots a collision, it broadcasts a negative acknowledgement (NACK), and each node then figures out when a common TDMA period starts based on its delay estimate and retransmits in its assigned slot, avoiding repeat collisions with the same interferer. Both the timeout and the TDMA slots include a fixed guard time $t_{guard} = 2t_d + t_{ACK}$. The protocol's phases and timing equations are spelled out in Section III of [1]; Section 3.1 identifies the specific quantities our proposed protocol modifies. Table 6 lays out the pros and cons.

Table 6: Advantages and limitations of ARQ_HMAC.

Advantages	**Limitations**
Maintains consistent throughput performance regardless of network load since TDMA retransmissions overcome the collisions that occur in entirely contention-based protocols, while random access remains efficient with light network loads.	Propagation delay is calculated only once and is not modified to reflect changes in the environment and movement of nodes. The fixed guard period of $2t_d + t_{ACK}$ inflates timeouts and TDMA slots, and TDMA retransmissions receive no acknowledgment.

2.3 Comparison of the Approaches

Table 7 gives a comparison of the schemes under consideration, while Figure 1 shows their position in the wider taxonomy. Up till now, delay awareness and collision-free retransmission have mostly been addressed independently. DOTS and Asym-TDMA are both delay aware but lack a formalized retransmission stage. ARQ_HMAC, in turn, ensures collision free retransmission but uses a fixed delay value. Current state-of-the-art protocols do not combine (i) collision-free retransmission, (ii) dynamic delay estimation, and (iii) time margins calculated based on delay variance. Our proposed Adaptive ARQ_HMAC aims at filling that gap.

Table 7: Comparison of the reviewed schemes.

Scheme	Year	Pros	Cons
Slotted FAMA [8]	2006	No data collisions, fair access	Long slots, handshake per packet
UWAN-MAC [9]	2007	Energy efficient	Repeated collisions at high load
T-Lohi [13]	2008	Cheap tone-based contention	Long contention rounds
DOTS [14]	2014	Delay map, concurrent transmissions, mobility	Complex, no retransmission phase
Asym-TDMA [6]	2018	Handles mobility and asymmetry	Periodic updates, idle slots
ARQ_HMAC [1]	2025	Stable throughput, collision-free retransmission	Static delay estimate, fixed guard
Adaptive ARQ_HMAC (proposed)	N/A	Continuous delay tracking, adaptive guard and slots, adaptive backoff	Needs clock synchronisation; TDMA retransmissions unacknowledged

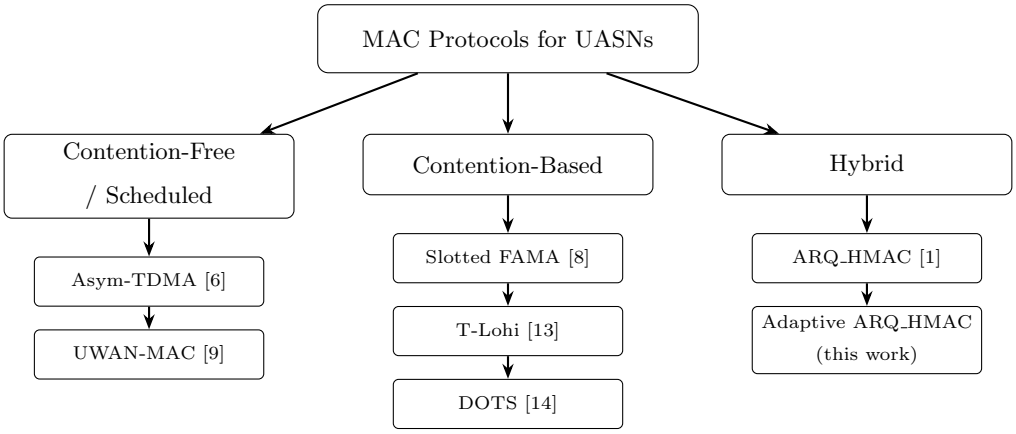


Figure 1: Classification of MAC protocols for underwater acoustic sensor networks.

3 Proposed Approach

3.1 Architecture

What the proposal changes. Our protocol leaves the overall structure of ARQ_HMAC untouched: the phases, frame types, and operating rules described in Section III of [1] all stay the same. That includes the initial phase (Eqs. (1)–(2) of [1]), the CSMA phase with its ACK/NACK exchange, the unacknowledged TDMA retransmission phase, and the TDMA period of $n\,t_{slot}$ (Eqs. (9)–(10) of [1]). The only things we touch are the quantities listed in Table 8. In the original ARQ_HMAC, the delay t_{D_i} gets measured a single time and then never updated. The guard interval $2t_d + t_{ACK}$ adds up to around a whole packet time's worth of additional overhead, and each TDMA timeslot also sets aside t_{max} in order to accommodate propagation delay spread. To put into perspective how much overhead this is: at a distance of 3 km, where $t_d = 1$ second and $t_{max} = 2$ seconds, one TDMA timeslot will be 4.1 seconds long, but only a second of that time contains any real information.

Table 8: Quantities of ARQ_HMAC that the proposed protocol changes; [1, Eq. k] refers to the equations of [1]. Everything else is unchanged.

Quantity	**ARQ_HMAC**	**Adaptive ARQ_HMAC**
Delay t_{D_i} [1, Eq. 2]	measured once in the initial phase	initial value from [1, Eq. 2], then $t_{D_i}^{sched}$ updated continuously, Eqs. (1)–(3)
Guard t_{guard} [1, Eq. 4]	$2t_d + t_{ACK}$	$t_{guard,i}$ from the measured deviation, Eq. (4)
ACK timeout [1, Eq. 3]	$2t_{D_i} + t_{guard}$	$2t_{D_i}^{sched} + t_{guard,i}$
TDMA start [1, Eq. 5]	uses t_{D_i}	uses $t_{D_i}^{sched}$
Slot width t_{slot} [1, Eq. 6]	$t_{max} + t_{guard}$	t_{guard}, set by the master and announced in the NACK
Slot start t_{slot_i} [1, Eq. 7]	transmission at slot start	arrival at slot start, Eq. (5)
Channel access	transmit as soon as the channel is free	adaptive random backoff

Proposed architecture. Here we present five novel methods that have been illustrated in Figure 2 below, namely: (1) adaptive delay estimation coupled with change detection, (2) delay measurement through timestamps in the master frames, (3) adaptive guard period, (4) aligned TDMA slots and (5) adaptive backoff. The first two methods cooperate in order to ensure that the estimate of the delay is always up-to-date. The rest use this to improve their performance.

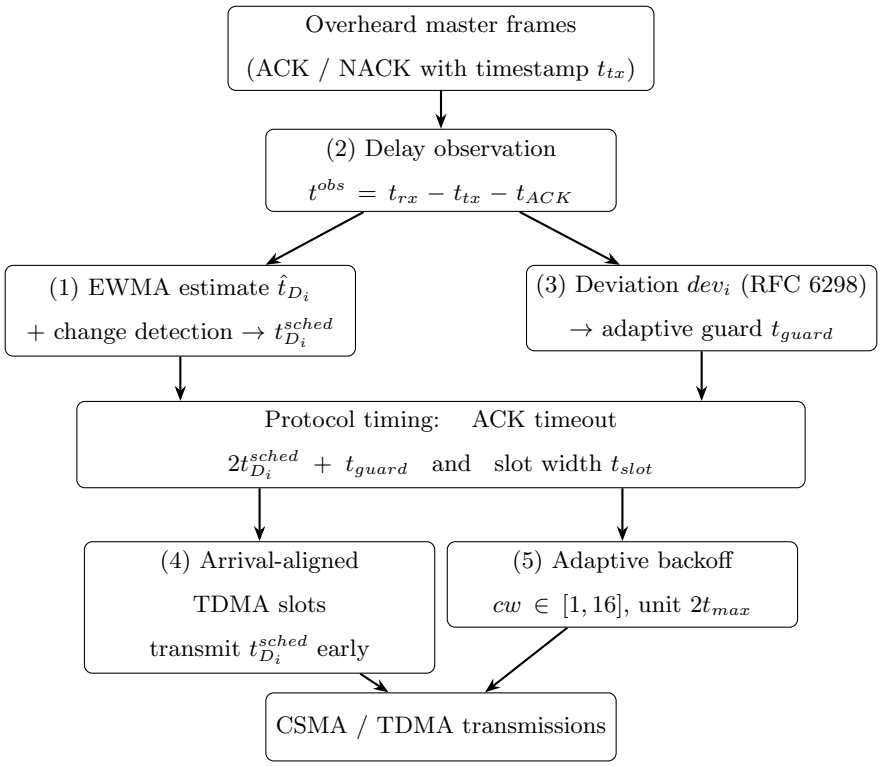


Figure 2: Architecture of Adaptive ARQ_HMAC. Blocks numbered (1)–(5) are the added mechanisms.

(1) Adaptive delay estimate with change detection. Building on what we outlined in the progress report, each sensor maintains a running estimate $\hat{t}_{D_i}$. We seed it with the value measured during the initial phase (Eq. (2) of [1]) and then refresh it every time a new observation comes in, using an Exponentially Weighted Moving Average (EWMA):

$$\hat{t}_{D_i}^{(k)} = \alpha \cdot t_{D_i}^{\mathrm{obs},(k)} + (1-\alpha) \cdot \hat{t}_{D_i}^{(k-1)}. \tag{1}$$

We do not want the protocol to recalculate its entire schedule every time the estimate jitters by a millisecond or two, so the value that actually feeds into the timing logic, $t_{D_i}^{\mathrm{sched}}$, only gets replaced when the drift crosses a threshold θ:

$$\Delta_i^{(k)} = \left| \hat{t}_{D_i}^{(k)} - t_{D_i}^{\mathrm{sched}} \right| > \theta \;\Rightarrow\; t_{D_i}^{\mathrm{sched}} \leftarrow \hat{t}_{D_i}^{(k)}, \tag{2}$$

at which point the ACK timeout and the TDMA schedule (Eqs. (3) and (5)–(7) of [1]) are recomputed using the fresh $t_{D_i}^{\text{sched}}$. In our setup we chose $\theta = 5$ ms and $\alpha = 0.7$. The relatively high α puts more weight on the latest observation, which makes sense for a node that may have moved a fair bit between two consecutive measurements.

(2) Timestamped delay observations. In the progress report version, a delay observation came from the round trip of a node's own data packet and its ACK: $t_{D_i}^{\text{obs}} = (t_{\text{ACK_recv}} - t_{\text{data_send}} - t_d - t_{ACK})/2$. The problem is that under moderate traffic a node might only see its own ACK once every minute or two, so the estimate falls behind quickly when the node is on the move. Fortunately, ARQ_HMAC already assumes that all nodes share a common time reference after the initial phase (the operating phase starts at $t = 0$ for everyone [1]). We take advantage of this by having the master stamp the transmission start time t_{tx} into every ACK and NACK it sends, and by broadcasting ACKs with the destination address tucked into the payload so that every sensor in range can overhear them. Whenever a sensor receives a master frame at time t_{rx} (measured at the end of reception), it can extract a one-way delay observation:

$$t_{D_i}^{\text{obs}} = t_{rx} - t_{tx} - t_{ACK}. \tag{3}$$

In our simulations this trick gives each sensor roughly 130 observations per 30-minute run at moderate load, compared to about one per own ACK. The idea is not new: DOTS [14] maintains its delay map in essentially the same way.

(3) Adaptive guard time. The guard interval has to cover the actual duration of the data and ACK frames ($t_d + t_{ACK}$) plus whatever uncertainty remains in the delay estimate. Rather than hard-coding an extra margin of one full data-packet time, we measure the uncertainty on the fly, borrowing the approach TCP uses to track round-trip time variation [16, 17]:

$$dev_i \leftarrow (1-\beta)\, dev_i + \beta \left| t_{D_i}^{\text{obs}} - \hat{t}_{D_i} \right|, \qquad t_{guard,i} = t_d + t_{ACK} + \max\left(t_{ACK},\ 2K\, dev_i\right), \tag{4}$$

using the standard RFC 6298 constants $\beta = 1/4$ and $K = 4$. We initialize the deviation at $dev_i = t_d/(2K)$, which conveniently reproduces the original paper's guard of $2t_d + t_{ACK}$ right out of the gate. From there, the guard only shrinks as the measurements confirm that the delay is behaving predictably; if the delay starts jumping around, dev_i climbs and the guard swells back toward the conservative ARQ_HMAC values. Since all nodes in a TDMA round must agree on the same slot size, each sensor piggybacks its dev_i inside its data packets, and the master picks a slot width large enough for the least predictable sensor and broadcasts that width in every NACK.

(4) Arrival-aligned TDMA slots. In the original ARQ_HMAC, every node fires off its retransmission at the very start of its assigned slot (Eq. (7) of [1]). The packet then takes up to t_{max} to

reach the master, which is why each slot has to reserve t_{max} for propagation spread. We flip this around: in our version, a slot defines the window during which the packet *arrives* at the master, and node i transmits $t_{D_i}^{\text{sched}}$ earlier to compensate (t_{start_TDMA} is still computed as in Eq. (5) of [1], just with $t_{D_i}^{sched}$ plugged in):

$$t_{arrive} = t_{start_TDMA} + t_{max}, \qquad t_{slot_i} = t_{arrive} + n_i\, t_{slot} - t_{D_i}^{\text{sched}}, \qquad t_{slot} = t_{guard}. \tag{5}$$

The upshot is that packets land back to back at the master and each slot shrinks by the full t_{max}: at 400 m the slot drops from 2.37 s to roughly 1.2 s, and at 3 km it goes from 4.1 s down to about 1.3 s. This only works because the delay estimate is accurate enough to trust.

(5) Adaptive backoff. When a TDMA round wraps up, every node that still has queued packets jumps back into CSMA at the same instant. Nodes that had been holding off because the channel was busy also rush to transmit the moment it clears. A synchronized transmission burst is achieved and, needless to say, a new set of collisions occurs. In order to destroy synchronization, each node will wait for a random time chosen uniformly from $[0,\ cw \cdot 2t_{max}]$. The unit $2t_{max}$ corresponds to the longest possible propagation time between any two sensors, which means the first node to start transmitting will be heard by the others before their backoff timers expire. The contention window cw follows the binary exponential backoff rule from IEEE 802.11 [18]: it doubles after every NACK or timeout, caps out at 16, and resets to 1 following a successful ACK.

3.2 Algorithm

Algorithm 2 describes only those parts that are added or changed in the sensor with ARQ_HMAC, while all the rest is done as described in [1]. This expands upon the Algorithm 1 described in the progress report. These new steps are incorporated into the control flow of the sensor as shown in Figure 3.

Algorithm 2 Changes to a sensor of ARQ_HMAC (sensor i)

Require: $t_{D_i}^{(0)}$ from the initial phase of ARQ_HMAC, α, θ, β, K

$\hat{t}_{D_i} \leftarrow t_{D_i}^{(0)}$; $t_{D_i}^{\text{sched}} \leftarrow t_{D_i}^{(0)}$; $dev_i \leftarrow t_d/(2K)$; $cw \leftarrow 1$ ▷ new state

for each master frame (ACK or NACK, any destination) received **do**

  $t^{\text{obs}} \leftarrow t_{rx} - t_{tx} - t_{ACK}$ ▷ Eq. (3)

  $dev_i \leftarrow (1-\beta)\, dev_i + \beta\, |t^{\text{obs}} - \hat{t}_{D_i}|$; update $t_{guard,i}$ ▷ Eq. (4)

  $\hat{t}_{D_i} \leftarrow \alpha\, t^{\text{obs}} + (1-\alpha)\, \hat{t}_{D_i}$ ▷ Eq. (1)

  if $|\hat{t}_{D_i} - t_{D_i}^{\text{sched}}| > \theta$ **then**

    $t_{D_i}^{\text{sched}} \leftarrow \hat{t}_{D_i}$ ▷ Eq. (2)

  end if

end for

changed: ACK timeout $= 2t_{D_i}^{\text{sched}} + t_{guard,i}$; TDMA start computed with $t_{D_i}^{\text{sched}}$

added: before a transmission after a busy channel or a TDMA period, wait $U(0, cw \cdot 2t_{max})$

added: on ACK $cw \leftarrow 1$; on NACK or timeout $cw \leftarrow \min(2cw, 16)$

changed: in TDMA, use the slot width announced in the NACK and transmit $t_{D_i}^{\text{sched}}$ before the arrival slot (Eq. 5)

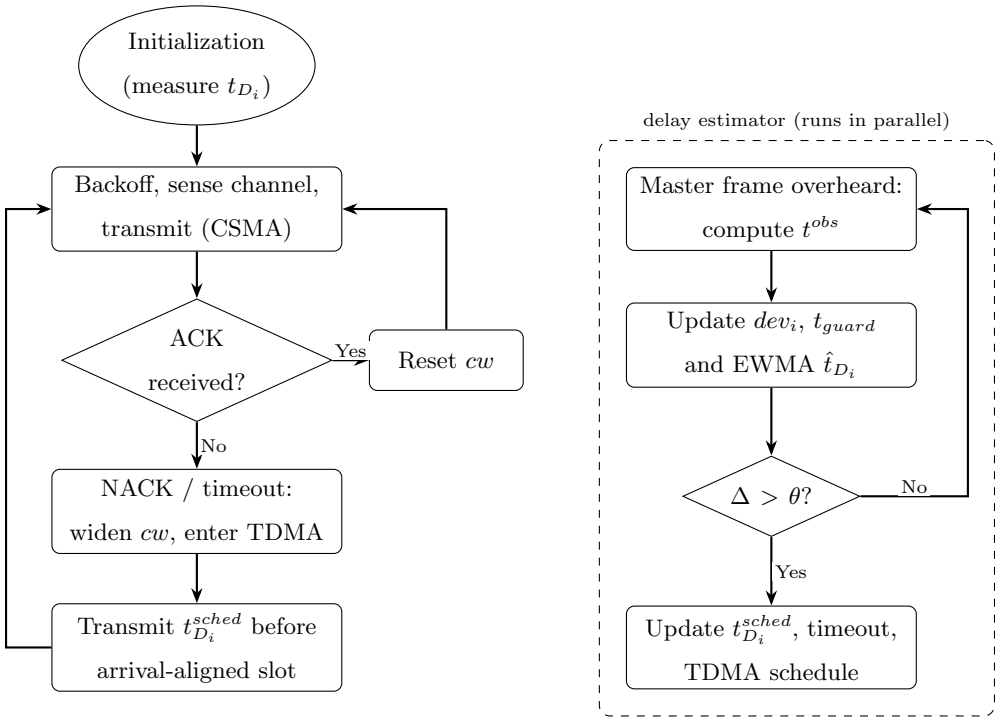


Figure 3: Flow of a sensor in Adaptive ARQ_HMAC. The delay estimator (right) runs in parallel with the transmission logic (left) and changes the timeout and schedule it uses.

3.3 Justification of Design Choices

- **EWMA with change detection.** The EWMA filter requires very little computation, just one multiplication and one addition per sample, and the threshold θ ensures that we do not

break our schedule every time the estimate wobbles by an amount too small to actually cause a timing error.

- **Observations using overheard master frames.** Since ACKs and NACKs are already being sent regardless, we get delay observations essentially for free, with no extra frames on the wire. All we add is a 4-byte timestamp and a single byte for either the deviation or the destination address to frames that already exist.
- **Guard proportional to the measured variation, beginning with the value in the paper.** The design is safe from the start. The guard will only get smaller when the measured variation is truly tiny. In case the delay becomes unpredictable, *dev* gets larger and the guard goes up again.
- **Arrival-aligned slots.** Aligning slots by when packets arrive rather than when they are transmitted strips t_{max} out of the per-slot calculation entirely. This matters most in large networks where t_{max} can easily exceed the packet transmission time.
- **Backoff with unit $2t_{max}$ and binary exponential window.** Setting the backoff unit to the maximum sensor-to-sensor propagation time means the randomization actually separates transmissions in a physically meaningful way. Doubling the window after errors lets the protocol adapt to whatever load the network is currently experiencing.
- **Standard, untuned constants.** The values of β, K, and the backoff rule come straight from RFC 6298 and IEEE 802.11, and the backoff unit is derived from the propagation physics. None of these were hand-tuned to our simulation results.

4 Implementation

4.1 Software Architecture in UnetStack

We built both protocols on top of UnetStack 7.0 [15], an agent-based protocol stack and discrete-event simulator for underwater networks, writing everything in Groovy. In UnetStack, each protocol lives inside an *agent* that communicates with other agents on the same node by passing messages. Our MAC agent talks directly to the physical-layer agent (`phy`): it fires off `TxFrameReq` messages when it wants to transmit, and it listens for `RxFrameNtf` notifications when a frame has been decoded successfully or `BadFrameNtf` notifications when decoding fails, which the master treats as evidence of a collision. The simulator handles the acoustic propagation delay based on the actual positions of the nodes (including any that are moving) and drops frames that overlap at a receiver. The protocol code lives in `arqhmac/ArqHmacAgent.groovy`, the network scenario in `arqhmac/Scenario.groovy`, the experiment runner in `run_comparison.groovy`, and the analysis scripts in `plot_results.py`.

Rather than maintaining two separate codebases, we implemented both protocols inside a single agent class, `ArqHmacAgent`, controlled by a boolean flag called `proposed`. When `proposed = false`,

the agent behaves exactly like the published ARQ_HMAC. When `proposed = true`, it activates the five mechanisms from Section 3. Keeping everything else identical means any performance gap we observe can be attributed squarely to the proposed mechanisms rather than to some unrelated implementation quirk. Figure 4 lays out the class structure; the values in parentheses are the default parameters.

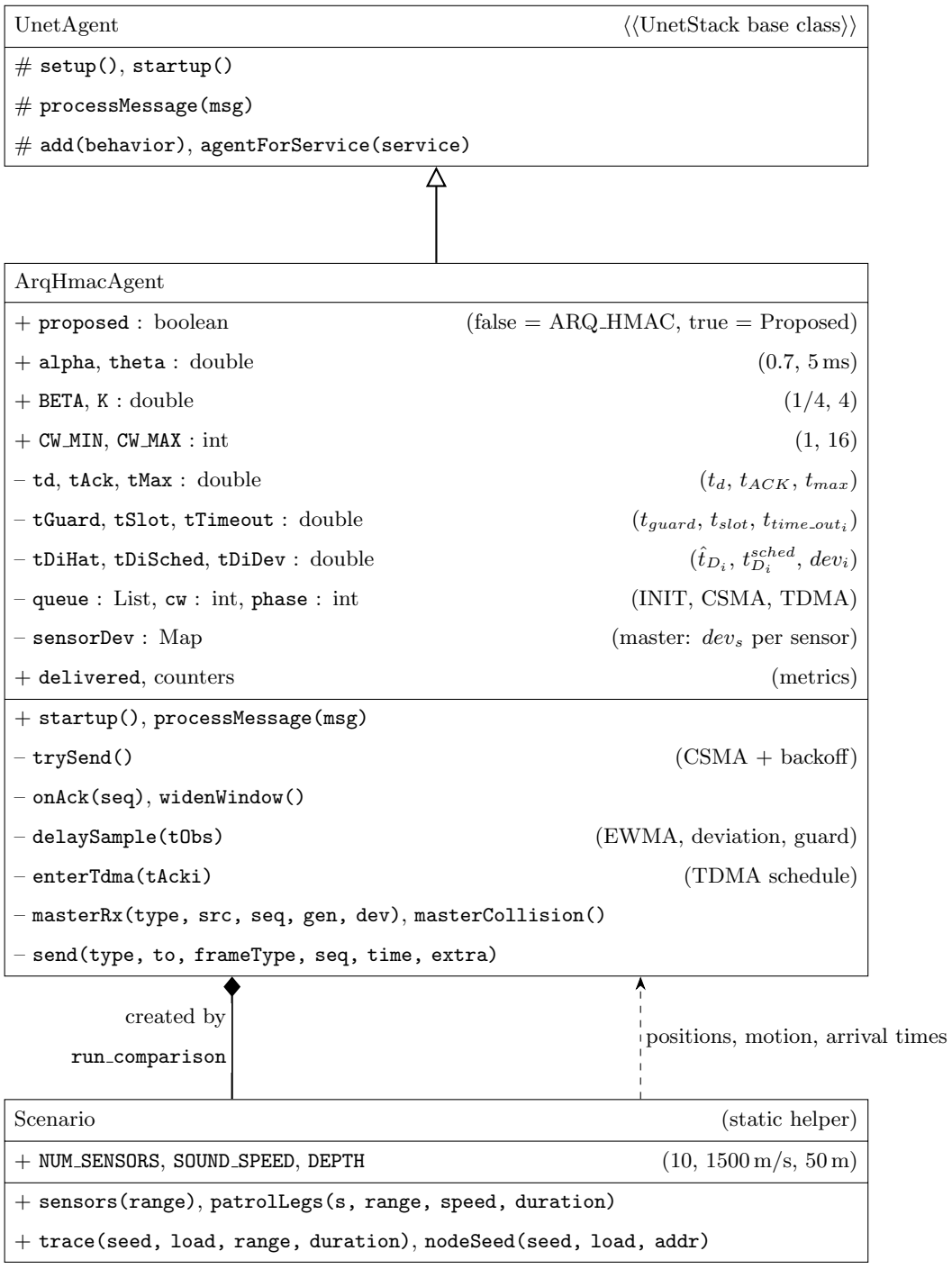


Figure 4: Class diagram of the implementation. Values in parentheses are the default parameters or the symbols used in Section 3.

4.2 Key Subsystems

Delay estimator. The `delaySample(tObs)` routine carries out the calculations described in Eqs. (1), (2) and (4). It first tosses out any observation that looks physically impossible ($t^{obs} < 0$ or $t^{obs} > 2t_{max}$), then updates the deviation before touching the mean (following the RFC 6298 ordering), recomputes the guard and timeout, and only swaps in a new $t_{D_i}^{sched}$ when the shift exceeds θ. This routine fires every time a sensor receives a master frame, regardless of who the frame was actually addressed to (see Algorithm 3).

Backoff in `trySend()`. The CSMA logic from ARQ_HMAC stays the same, with one twist: a node that found the channel busy, or one that just exited a TDMA period, gets flagged as *deferred* and has to wait a random interval before it can transmit. The contention window *cw* gets adjusted whenever an ACK, NACK, or timeout occurs (Algorithm 4).

Slots in `enterTdma()`. The TDMA phase follows ARQ_HMAC as well, except that the transmission timing and the end of the TDMA period now use the arrival-aligned slot formula from Eq. (5), with whatever slot width the master announced (Algorithm 4).

Master. The master node runs ARQ_HMAC with a few additions: it stamps its ACKs and NACKs with the transmission time, records the deviation value piggybacked on each incoming data packet, and picks a slot width large enough for the worst-case sensor before announcing it in the NACK (Algorithm 4).

Frame format. Every frame carries an 8-byte payload consisting of the frame type (INIT, DATA, ACK, NACK, or TDMA_DATA), a 16-bit sequence number, a 32-bit time field in milliseconds, and one spare byte. In data frames the time field stores the packet's generation time (which we later use to compute end-to-end delay) and the extra byte holds the deviation dev_i. In master frames the time field carries the transmission start time for delay observations. ACKs use the extra byte for the destination address, while NACKs repurpose the sequence field to announce the TDMA slot width. The modem is set up so that a data frame takes $t_d = 1$ s to transmit and a control frame takes $t_{ACK} = 0.1$ s.

4.3 Message Processing Path

Figure 5 traces how `processMessage` handles each notification coming up from the physical layer; the step tagged [P] only exists in the proposed protocol. Algorithm 3 details the new delay estimator and Algorithm 4 lists the modifications to the existing ARQ_HMAC handlers. The parts of the handlers that we did not change follow [1] exactly and are not reproduced here.

Algorithm 3 Delay observation (`delaySample`) [P]

Require: observation t^{obs}

Ensure: updated $\hat{t}_{D_i}$, dev_i, $t_{guard,i}$, $t_{D_i}^{sched}$, $t_{time_out_i}$

if $t^{obs} < 0$ **or** $t^{obs} > 2t_{max}$ **then**
  return ▷ implausible sample
end if
$dev_i \leftarrow (1-\beta)\, dev_i + \beta\, |t^{obs} - \hat{t}_{D_i}|$ ▷ deviation before mean
$t_{guard,i} \leftarrow t_d + t_{ACK} + \max(t_{ACK}, 2K\, dev_i)$; $\quad t_{time_out_i} \leftarrow 2t_{D_i}^{sched} + t_{guard,i}$
$\hat{t}_{D_i} \leftarrow \alpha\, t^{obs} + (1-\alpha)\, \hat{t}_{D_i}$
if $|\hat{t}_{D_i} - t_{D_i}^{sched}| > \theta$ **then**
  $t_{D_i}^{sched} \leftarrow \hat{t}_{D_i}$; $\quad t_{time_out_i} \leftarrow 2t_{D_i}^{sched} + t_{guard,i}$
end if

Algorithm 4 Modifications to the ARQ_HMAC handlers (only the changed lines)

trySend, channel busy: deferred $\leftarrow$ true
trySend, channel free and deferred: deferred $\leftarrow$ false; wait $U(0,\, cw \cdot 2t_{max})$, then retry
trySend, sending DATA: add dev_i to the frame; timer $\leftarrow 2t_{D_i}^{sched} + t_{guard,i}$
onAck: $cw \leftarrow 1$
on timeout or NACK for own packet: $cw \leftarrow \min(2cw, 16)$; on NACK $t_{slot} \leftarrow$ announced width
enterTdma: $t_{start} \leftarrow t_{ACK_i} + t_{max} - t_{D_i}^{sched}$; $\quad t_{tx} \leftarrow t_{start} + t_{max} + n_i t_{slot} - t_{D_i}^{sched}$; $\quad t_{end} \leftarrow t_{start} + t_{max} + n\, t_{slot}$
end of TDMA: deferred $\leftarrow$ true
master, on DATA: $dev_s \leftarrow$ reported deviation; ACK carries transmit time and destination
master, on collision: $t_{slot} \leftarrow t_d + t_{ACK} + \max(t_{ACK}, 2K \max_s dev_s)$; NACK carries t_{slot} and transmit time

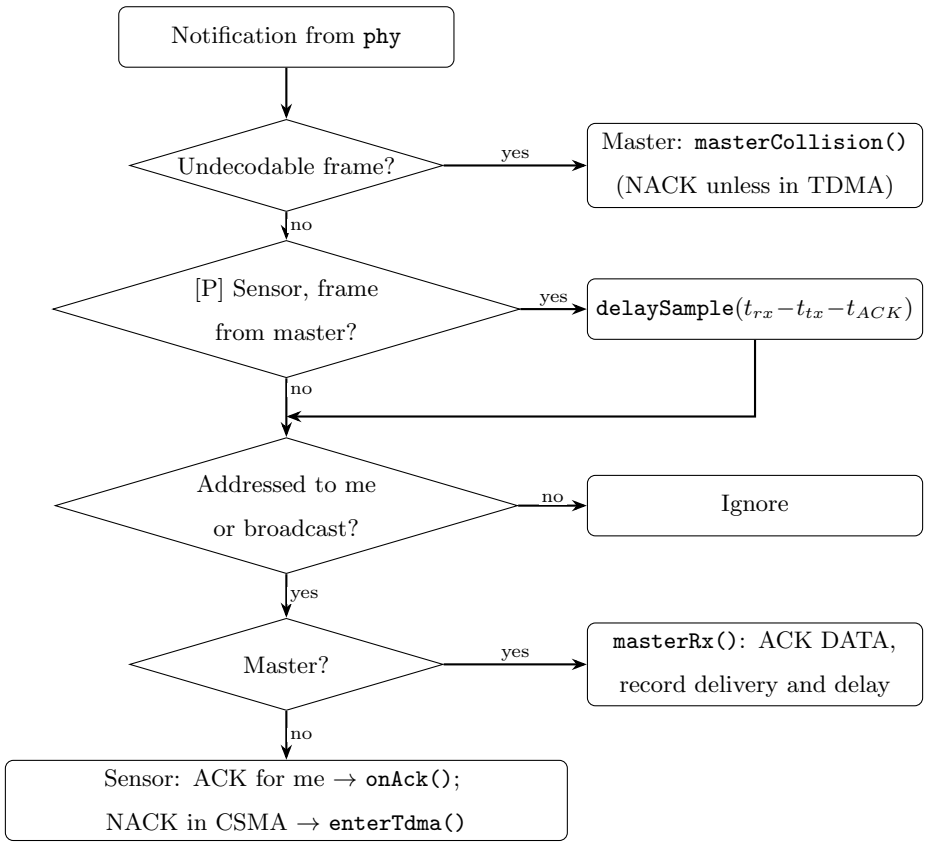


Figure 5: Flow of `processMessage` for every notification from the physical layer.

4.4 Complexity and Overhead

- **Computations.** Each delay observation boils down to a handful of $O(1)$ arithmetic operations. The master has to find the maximum deviation across all reported DATA packets once per NACK, which takes $O(n)$ operations for n sensors, hardly a bottleneck.
- **Memory.** A sensor needs to stash five extra numbers ($\hat{t}_{D_i}$, $t_{D_i}^{sched}$, dev_i, $t_{guard,i}$, cw). The master keeps one deviation value per sensor. The footprint is negligible.
- **Communication.** We did not introduce any new frame types. The timestamp and the extra byte squeeze into the existing 8-byte payload, so control frames stay at their original duration of $t_{ACK} = 0.1$ s.

4.5 Configurable Parameters

The values of the protocol parameters are outlined in Table 9. These were finalized prior to experimentation on the basis of existing standards and fundamental considerations. The different tuning seeds (101–105) were utilized for determining the inclusion/exclusion of certain mechanisms in general, and not individual parameters.

Table 9: Protocol parameters used in the experiments.

Parameter	Meaning	Value
α	EWMA smoothing factor	0.7
θ	Change-detection threshold	5 ms
β, K	Deviation gain, guard multiplier (RFC 6298)	1/4, 4
$dev_i^{(0)}$	Initial deviation	$t_d/(2K)$
t_{guard}	Guard (baseline / proposed)	$2t_d + t_{ACK}$ / Eq. (4)
cw	Contention window, unit $2t_{max}$	1 … 16
–	Carrier re-check interval	10 ms

Deviations from the paper. The original paper defines the slot index as $n_i = \lfloor t_{D_i}/t_{slot} \rfloor$. Since $t_{D_i} < t_{max} < t_{slot}$ in practice, this expression always evaluates to zero, which would dump every node into the same slot. Both of our implementations sidestep this by assigning each sensor its own slot based on its index in the network, which is consistent with the TDMA period of $n\,t_{slot}$. We also broadcast ACKs with the destination embedded in the payload in both protocol variants; this does not alter ARQ_HMAC's behaviour in any meaningful way. While implementing this solution, we tried out another mechanism which was a block acknowledgment for TDMA retransmissions on the tuning seeds; this was effective only at light loads and led to congestion collapse during heavy loads, hence we abandoned it.

5 Results and Analysis

5.1 Simulation Setup

We ran both protocols in UnetStack 7.0 using the `ProtocolChannelModel`. The master node sits moored in the middle of the deployment area. Around it are four moored sensors at distances between $0.38R$ and $0.75R$, plus six AUV sensors. Each AUV moves back and forth along a radial path between $0.2R$ and $0.85R$, with its own bearing and starting position, so its delay to the master is always changing. Every sensor can reach the master, but sensors on opposite sides cannot hear one another, which produces hidden terminals much like those in a real deployment. We consider three channel conditions (Table 10); the remaining parameters appear in Table 11. For each combination of condition, load and seed we generate a single traffic trace and feed it to both protocols. The script also verifies that the two protocols produced exactly the same set of packets.

Table 10: Channel conditions.

Condition	Range R	AUV speed	One-way delay change
Static-400m	400 m	0 (all fixed, paper setup)	none
Mobile-400m	400 m	0.3 m/s	up to 0.17 s
Mobile-3km	3000 m	2 m/s	up to 1.3 s

Table 11: Simulation parameters.

Parameter	**Value**
Simulator	UnetStack 7.0, `ProtocolChannelModel`
Nodes	1 master + 10 sensors (4 moored, 6 AUVs), depth 50 m
Sound speed	1500 m/s
Data rate	1200 bps
Data / control packet	1200 bits ($t_d = 1$ s) / 120 bits ($t_{ACK} = 0.1$ s)
Communication range R	400 m ($t_{max} = 0.27$ s) or 3000 m ($t_{max} = 2$ s)
Traffic	Poisson, offered load $G \in \{0.05, 0.1, 0.2, 0.3\}$
Duration	1800 s per run
Repetitions	10 seeds per (condition, load), 240 runs in total
Compared protocols	ARQ_HMAC, Adaptive ARQ_HMAC

We define the offered load G as the number of newly generated packets per data-packet duration across the entire network. For example, $G = 0.1$ corresponds to one new packet every 10 seconds, shared among the ten sensors. The modem settings are taken from Table 2 of the ARQ_HMAC paper [1].

5.2 Performance Metrics

- **Packet Delivery Ratio (PDR)**: the number of distinct packets that reached the master, divided by the number generated.
- **Throughput** S: delivered packets multiplied by t_d and divided by the operating time. This is the share of time the channel spends carrying successful data, following the definition in the paper.
- **Number of retransmissions**: packets that were resent in a TDMA slot after their CSMA attempt failed.
- **Collision rate**: collisions detected at the master divided by the frames the sensors sent.
- **End-to-end delay**: the time between a packet being generated and its first reception at the master.
- **Delay estimate error** (diagnostic): the mean absolute gap between the delay value a sensor is using and its true delay derived from its position, sampled every 10 s.

Since both protocols receive identical traffic, we compare their runs pairwise using the two-sided Wilcoxon signed-rank test (40 pairs per condition). A difference counts as significant when $p < 0.05$.

5.3 Numerical Results

Table 12 gives the outcome of all 240 simulations, averaged over the four loads and ten seeds. The proposed protocol is better on every metric in every condition, and each difference is statistically

significant ($p < 0.001$). The "B/E/W" column gives the number of paired runs in which the proposed protocol was better, equal or worse than ARQ_HMAC. Table 13 splits the results by offered load.

Table 12: Overall results (mean over 4 loads × 10 seeds; all $p < 0.001$).

Condition	Metric	ARQ_HMAC	Proposed	Change	B/E/W
Static-400m	PDR (%)	93.08	**96.30**	+3.5%	22/13/5
	Throughput S	0.144	**0.153**	+6.5%	22/13/5
	Retransmissions	138.1	**75.9**	−45.1%	36/4/0
	Collision rate (%)	10.37	**7.92**	−23.6%	27/3/10
	End-to-end delay (s)	12.01	**3.96**	−67.0%	39/0/1
Mobile-400m	PDR (%)	92.72	**95.92**	+3.4%	22/10/8
	Throughput S	0.143	**0.152**	+6.3%	22/10/8
	Retransmissions	134.2	**75.4**	−43.8%	36/3/1
	Collision rate (%)	10.42	**8.06**	−22.7%	27/2/11
	End-to-end delay (s)	11.26	**3.93**	−65.1%	39/0/1
Mobile-3km	PDR (%)	79.46	**89.34**	+12.4%	30/8/2
	Throughput S	0.110	**0.135**	+22.7%	30/8/2
	Retransmissions	176.9	**94.3**	−46.7%	40/0/0
	Collision rate (%)	17.42	**12.22**	−29.9%	39/1/0
	End-to-end delay (s)	89.37	**25.11**	−71.9%	40/0/0

Table 13: Results per offered load (ARQ_HMAC → Proposed, mean over 10 seeds).

Condition	G	PDR (%)	Throughput S	Retransmissions	Collisions (%)	Delay (s)
Static-400m	0.05	99.8 → 99.9	0.050 → 0.050	6.7 → 4.2	2.0 → 1.9	2.0 → 1.7
	0.10	99.2 → 99.4	0.098 → 0.098	27.0 → 19.9	4.7 → 4.2	3.5 → 2.4
	0.20	94.2 → 95.9	0.188 → 0.191	153.3 → 86.2	13.2 → 10.2	12.3 → 4.4
	0.30	79.1 → 90.0	0.241 → 0.275	365.5 → 193.2	21.5 → 15.3	30.3 → 7.4
Mobile-400m	0.05	99.9 → 100.0	0.050 → 0.050	6.5 → 4.4	1.7 → 1.7	2.0 → 1.7
	0.10	99.1 → 98.7	0.098 → 0.097	27.0 → 18.3	4.9 → 4.6	3.5 → 2.2
	0.20	93.1 → 96.5	0.185 → 0.192	149.0 → 80.9	13.2 → 9.5	11.8 → 4.2
	0.30	78.8 → 88.4	0.240 → 0.270	354.4 → 198.0	21.9 → 16.4	27.7 → 7.5
Mobile-3km	0.05	99.7 → 100.0	0.050 → 0.050	23.1 → 9.2	6.0 → 3.3	8.0 → 4.1
	0.10	95.5 → 97.3	0.094 → 0.096	79.3 → 37.0	12.7 → 8.3	20.4 → 7.0
	0.20	72.6 → 88.3	0.144 → 0.176	260.0 → 131.8	23.7 → 15.9	91.9 → 18.6
	0.30	50.0 → 71.8	0.152 → 0.219	345.2 → 199.2	27.3 → 21.4	237.2 → 70.7

5.4 Graphical Analysis

Delay tracking. Figure 6 illustrates the main idea of the proposal for a single AUV over one complete run. The true delay of this vehicle varies between 55 ms and 225 ms. ARQ_HMAC stays with the 135 ms value it measured at start-up and is therefore off by as much as 90 ms, whereas the proposed protocol stays within a few milliseconds of the real value. Averaged over all sensors, the delay estimate error drops from 36.3 ms to 3.0 ms in Mobile-400m and from 306 ms to 15 ms in Mobile-3km, which is a reduction of 92% and 95% respectively. In the static network both protocols measure the delay only once, so their estimates are the same.

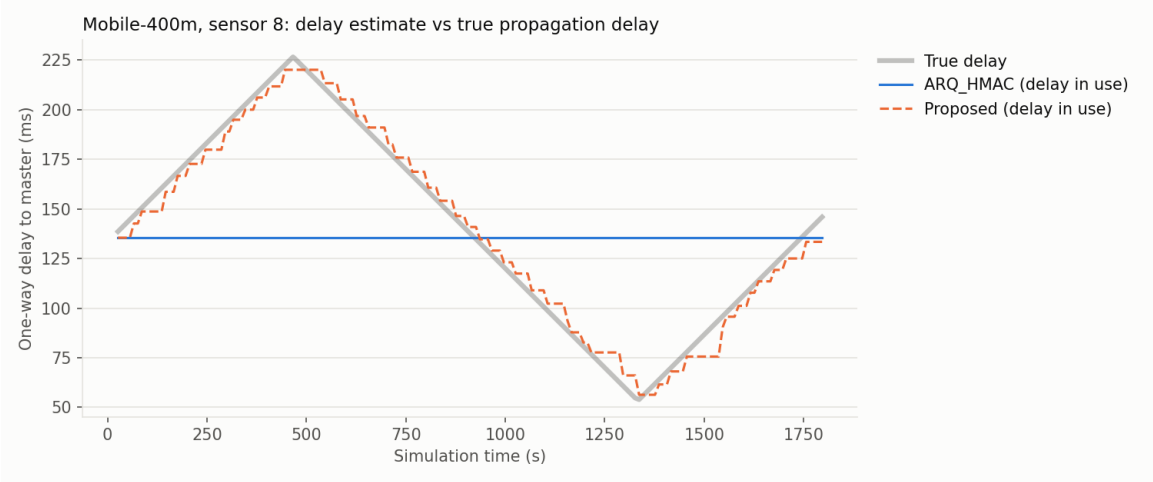


Figure 6: Delay value in use versus true propagation delay of one AUV (Mobile-400m, $G = 0.1$). ARQ_HMAC keeps its start-up measurement; the proposed protocol follows the moving AUV.

End-to-end delay. End-to-end delay improves the most dramatically by reducing the end-to-end delay in 65–72% under all conditions (see Figures 7–9). When the guard is correctly estimated, it may be reduced from 2.1 s to roughly 1.2–1.4 s, and the arrival-aligned slots take t_{max} out of each slot. As a result, a TDMA period for ten sensors lasts 12–16 s, compared with 24 s at 400 m and 41 s at 3 km for the baseline. Retransmitted packets get to the master earlier, and the channel goes back to CSMA sooner. The improvement grows with load, since a larger share of packets passes through TDMA periods and queues start to build.

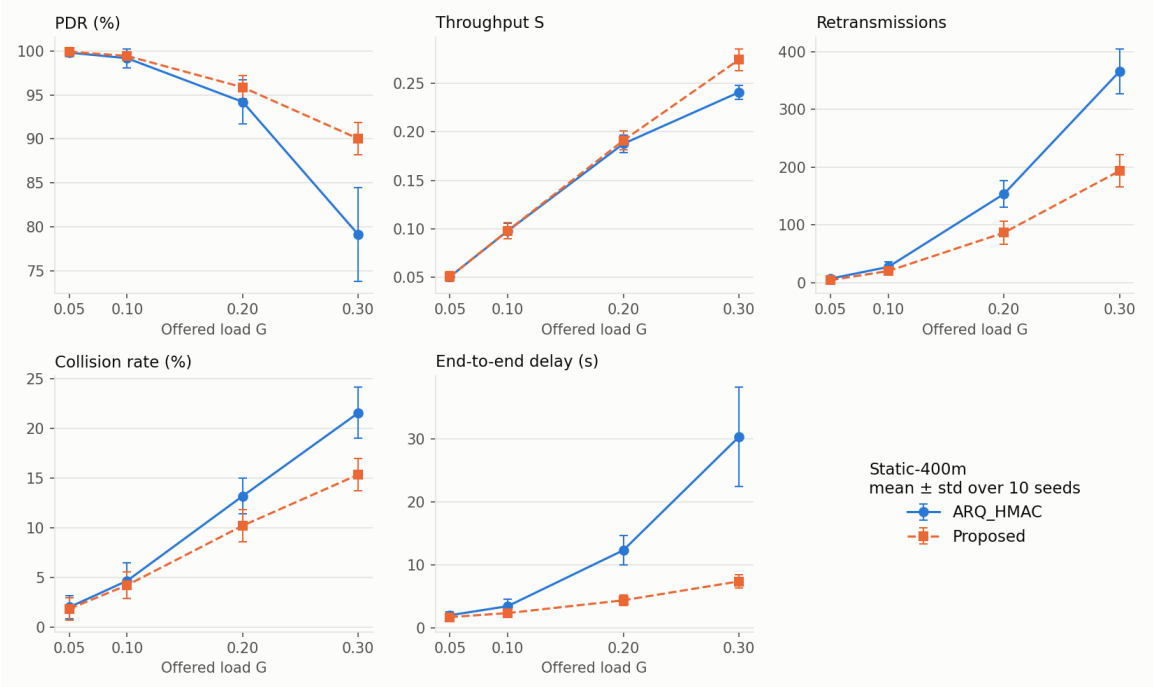


Figure 7: Static-400m: PDR, throughput, retransmissions, collision rate and end-to-end delay versus offered load.

Retransmissions and collisions. The number of retransmissions goes down by 44–47% and the collision rate by 23–30%. Adaptive backoff desynchronises nodes that finish a TDMA period at the same moment or that wait on the same busy channel. This eliminates many of the collisions that would otherwise trigger another TDMA period. In addition, the shorter timeouts let a node notice a lost packet sooner.

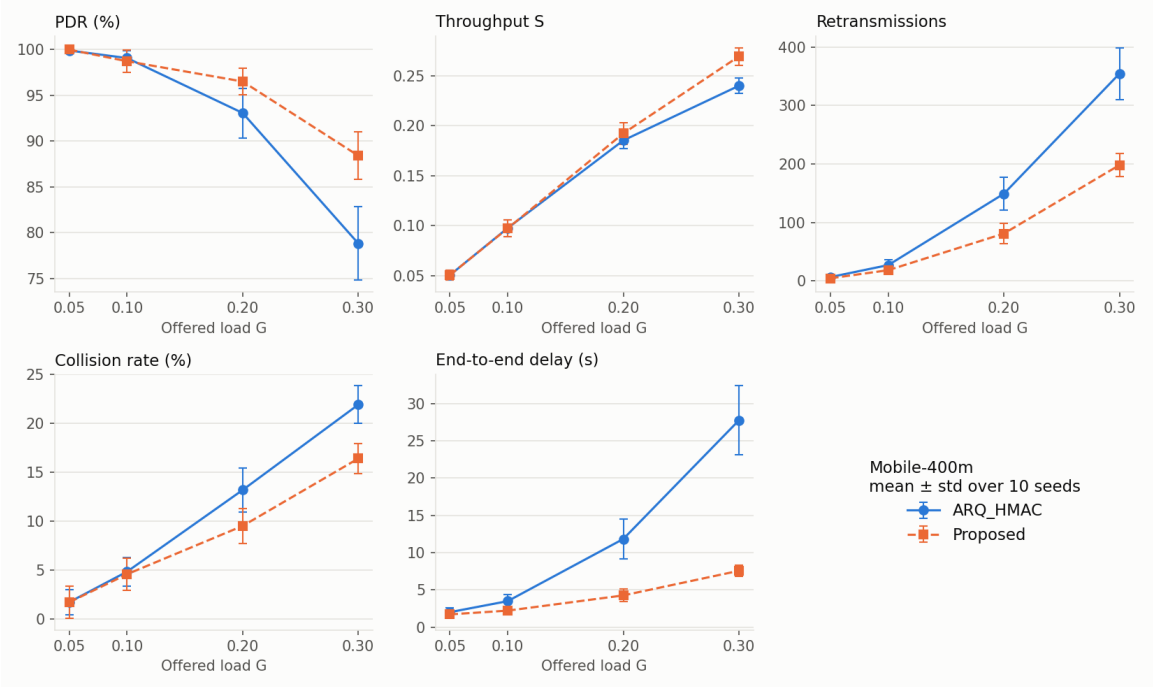


Figure 8: Mobile-400m: PDR, throughput, retransmissions, collision rate and end-to-end delay versus offered load.

PDR and throughput. When the load is light, the channel is seldom contended and both protocols deliver nearly everything. At high load, the long TDMA periods and frequent collisions of the baseline lead to losses, and at $G = 0.3$ its PDR falls to 79% at 400 m and 50% at 3 km. The proposed protocol holds PDR at 88–90% at 400 m and 72% at 3 km, and its throughput continues to climb with load while the baseline levels off (Figure 9). The 3 km network gains the most (+22.7% throughput, +12.4% PDR), since t_{max} is largest there and so a precise delay value matters most.

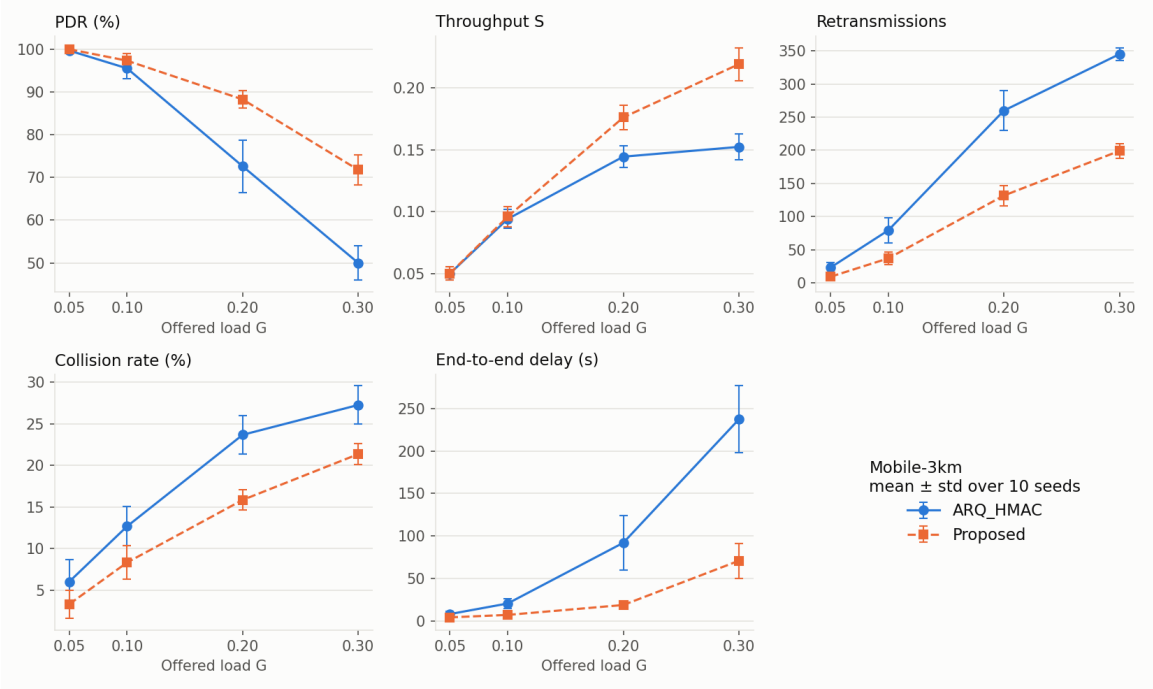


Figure 9: Mobile-3km: PDR, throughput, retransmissions, collision rate and end-to-end delay versus offered load.

Static network. The proposed protocol is significantly better even when no node moves. The measured delay variation is small, so the adaptive guard and aligned slots can shorten the timing without risk. The gain is therefore not only a result of mobility: the fixed margins in the paper are conservative, and tracking the delay continuously makes it possible to trim them.

5.5 Comparison with the State of the Art

Among hybrid ARQ-based MAC protocols for UASNs, ARQ_HMAC is the most recent, and its authors reported higher and steadier throughput than Pure ALOHA, ALOHA with CSMA, Slotted ALOHA and Slotted ALOHA with CSMA [1]. Table 14 compares the reviewed schemes at the design level, and the bullets below connect those design differences to our measurements.

Table 14: Design level comparison with the state of the art.

Protocol	Collision-free retransmission	Delay aware	Continuous delay update	Adaptive timing margin	Adaptive contention
Slotted FAMA [8]	No	No	No	No	No
T-Lohi [13]	No	No	No	No	Yes
DOTS [14]	No	Yes	Yes	No	No
Asym-TDMA [6]	Yes	Yes	Periodic	No	No
ARQ_HMAC [1]	Yes	Yes	No	No	No
Adaptive ARQ_HMAC	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

- **Versus ARQ_HMAC.** With identical traffic and node motion, the proposed protocol gives a significant improvement on all five metrics in the static network, in the mobile network of

the size used in the paper, and in the long-range mobile network. Delay is 65–72% lower, retransmissions are 44–47% fewer, collisions are 23–30% fewer, throughput is 6–23% higher and PDR is 3–12% higher.

- **Versus the protocols ARQ_HMAC was compared with.** In the paper's evaluation ARQ_HMAC beats the ALOHA and CSMA variants, and here the proposed protocol beats ARQ_HMAC. We therefore expect it to beat those variants too, but we did not simulate them again.

- **Versus delay-aware protocols.** Like DOTS, the proposed protocol learns delays from overheard, timestamped traffic. It differs by keeping the collision-free retransmission phase of ARQ_HMAC and using the delay knowledge to make that phase shorter. Compared with Asym-TDMA, it updates the delay continuously rather than at fixed intervals. These comparisons rest on design alone, because DOTS and Asym-TDMA were not simulated.

5.6 Discussion and Limitations

The experiments show that continuous delay tracking combined with adaptive timing improves ARQ_HMAC under every condition we tested, most clearly in the large network and at heavy load. At light load ($G \leq 0.1$) the differences in PDR and throughput stay within $\pm 2\%$, and in one case (Mobile-400m, $G = 0.1$) PDR is marginally lower (98.7% versus 99.1%). As in the paper, TDMA retransmissions are not acknowledged, so a frame lost in a slot stays lost in both protocols. The timestamped observations rely on synchronised clocks. These are perfect in the simulator, but a real deployment would need periodic resynchronisation. Lastly, the protocol channel model has no bit errors, multipath or sound-speed variation, so all delay changes in the simulation come from node motion alone.

6 Conclusions and Future Work

This work set out to remove a major weakness of ARQ_HMAC. Its retransmission scheduling depends on a propagation delay that is measured once at start-up and then covered by large fixed timing margins, even though underwater delays change with conditions and node movement. We proposed Adaptive ARQ_HMAC, which refreshes the delay estimate continuously with an EWMA filter and change detection, takes observations from timestamped master frames, derives its guard time from the measured delay variation, aligns TDMA slots with packet arrivals, and adjusts its backoff to the level of contention.

We implemented both protocols in UnetStack and compared them on identical traffic in 240 simulations. The proposed protocol follows the true delay with a mean error of 3 ms at 400 m and 15 ms at 3 km. Across the static, mobile and long-range scenarios it lowers end-to-end delay by

65–72%, retransmissions by 44–47% and collisions by 23–30%, and it raises throughput by 6–23% and packet delivery ratio by 3–12%. All of these differences are statistically significant.

Future work includes a stable acknowledgement scheme for TDMA retransmissions and clock-drift compensation for the timestamped observations. It also includes more realistic channel models with bit errors, multipath and sound-speed variation, energy measurements, and tests on real acoustic modems.

References

[1] V.-P. Dao and T.-H. Tran, "ARQ_HMAC: An automatic repeat request strategy for hybrid MAC protocol in underwater acoustic sensor networks," *IEEE Access*, vol. 13, pp. 159196–159206, 2025, doi: 10.1109/ACCESS.2025.3607746.

[2] I. F. Akyildiz, D. Pompili and T. Melodia, "Underwater acoustic sensor networks: Research challenges," *Ad Hoc Networks*, vol. 3, no. 3, pp. 257–279, 2005, doi: 10.1016/j.adhoc.2005.01.004.

[3] P. Casari and M. Zorzi, "Protocol design issues in underwater acoustic networks," *Computer Communications*, vol. 34, no. 17, pp. 2013–2025, 2011, doi: 10.1016/j.comcom.2011.06.008.

[4] S. Jiang, "State-of-the-art medium access control (MAC) protocols for underwater acoustic networks: A survey based on a MAC reference model," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 1, pp. 96–131, 2018, doi: 10.1109/COMST.2017.2768802.

[5] E. Kulla, K. Matsuo and L. Barolli, "MAC layer protocols for underwater acoustic sensor networks: A survey," *Lecture Notes in Networks and Systems*, vol. 496, pp. 211–220, 2022, doi: 10.1007/978-3-031-08819-3_21.

[6] A.-R. Cho, C. Yun, Y.-K. Lim and Y. Choi, "Asymmetric propagation delay-aware TDMA MAC protocol for mobile underwater acoustic sensor networks," *Applied Sciences*, vol. 8, no. 6, p. 962, 2018, doi: 10.3390/app8060962.

[7] D. J. Molinares and M. A. Labrador, "An adaptive logical link layer protocol for underwater acoustic communication channels," in *Proc. OCEANS 2009*, 2009, pp. 1–8, doi: 10.23919/OCEANS.2009.5422348.

[8] M. Molins and M. Stojanovic, "Slotted FAMA: A MAC protocol for underwater acoustic networks," in *Proc. IEEE OCEANS 2006 Asia Pacific*, 2006, pp. 1–7, doi: 10.1109/OCEANSAP.2006.4393832.

[9] M. K. Park and V. Rodoplu, "UWAN-MAC: An energy-efficient MAC protocol for underwater acoustic wireless sensor networks," *IEEE Journal of Oceanic Engineering*, vol. 32, no. 3, pp. 710–720, 2007, doi: 10.1109/JOE.2007.899277.

[10] M. Stojanovic, "Optimization of a data link protocol for an underwater acoustic channel," in *Proc. IEEE OCEANS 2005 Europe*, 2005, vol. 1, pp. 68–73, doi: 10.1109/OCEANSE.2005.1511686.

[11] N. Chirdchoo, W.-S. Soh and K. C. Chua, "Aloha-based MAC protocols with collision avoidance for underwater acoustic networks," in *Proc. IEEE INFOCOM*, 2007, pp. 2271–2275, doi: 10.1109/INFCOM.2007.263.

[12] B. Peleato and M. Stojanovic, "Distance aware collision avoidance protocol for ad-hoc underwater acoustic sensor networks," *IEEE Communications Letters*, vol. 11, no. 12, pp. 1025–1027, 2007, doi: 10.1109/LCOMM.2007.071160.

[13] A. A. Syed, W. Ye and J. Heidemann, "T-Lohi: A new class of MAC protocols for underwater acoustic sensor networks," in *Proc. IEEE INFOCOM*, 2008, pp. 231–235, doi: 10.1109/INFOCOM.2008.55.

[14] Y. Noh, U. Lee, S. Han, P. Wang, D. Torres, J. Kim and M. Gerla, "DOTS: A propagation delay-aware opportunistic MAC protocol for mobile underwater networks," *IEEE Transactions on Mobile Computing*, vol. 13, no. 4, pp. 766–782, 2014, doi: 10.1109/TMC.2013.2297703.

[15] M. Chitre, R. Bhatnagar and W.-S. Soh, "UnetStack: An agent-based software stack and simulator for underwater networks," in *Proc. IEEE OCEANS 2014*, St. John's, 2014, pp. 1–10, doi: 10.1109/OCEANS.2014.7003044.

[16] V. Jacobson, "Congestion avoidance and control," in *Proc. ACM SIGCOMM*, 1988, pp. 314–329, doi: 10.1145/52325.52356.

[17] V. Paxson, M. Allman, J. Chu and M. Sargent, "Computing TCP's retransmission timer," RFC 6298, Internet Engineering Task Force, Jun. 2011. Available: `https://www.rfc-editor.org/rfc/rfc6298.html`.

[18] IEEE, "IEEE standard for information technology – Wireless LAN medium access control (MAC) and physical layer (PHY) specifications," IEEE Std 802.11-2016, 2016, doi: 10.1109/IEEESTD.2016.7786995.